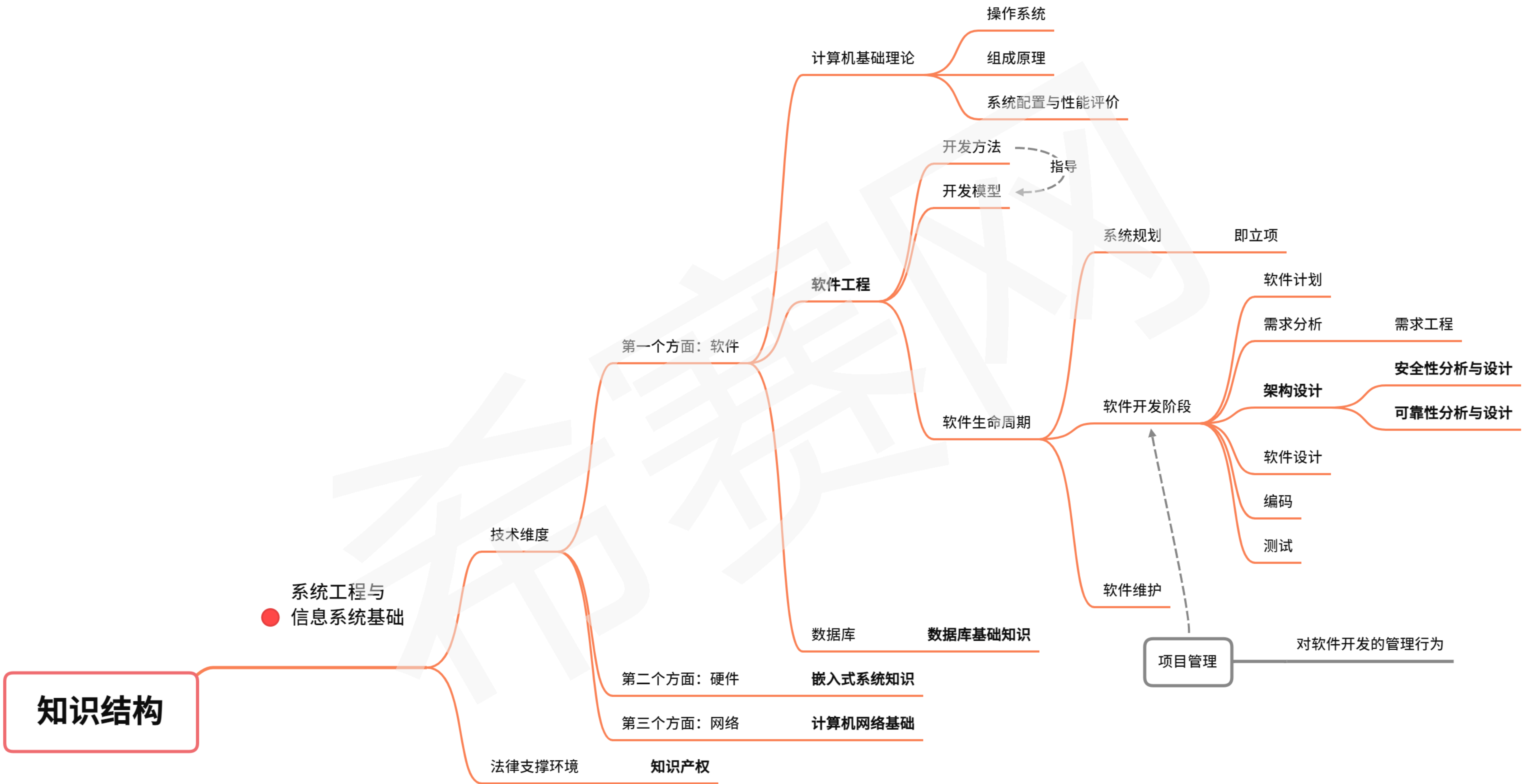
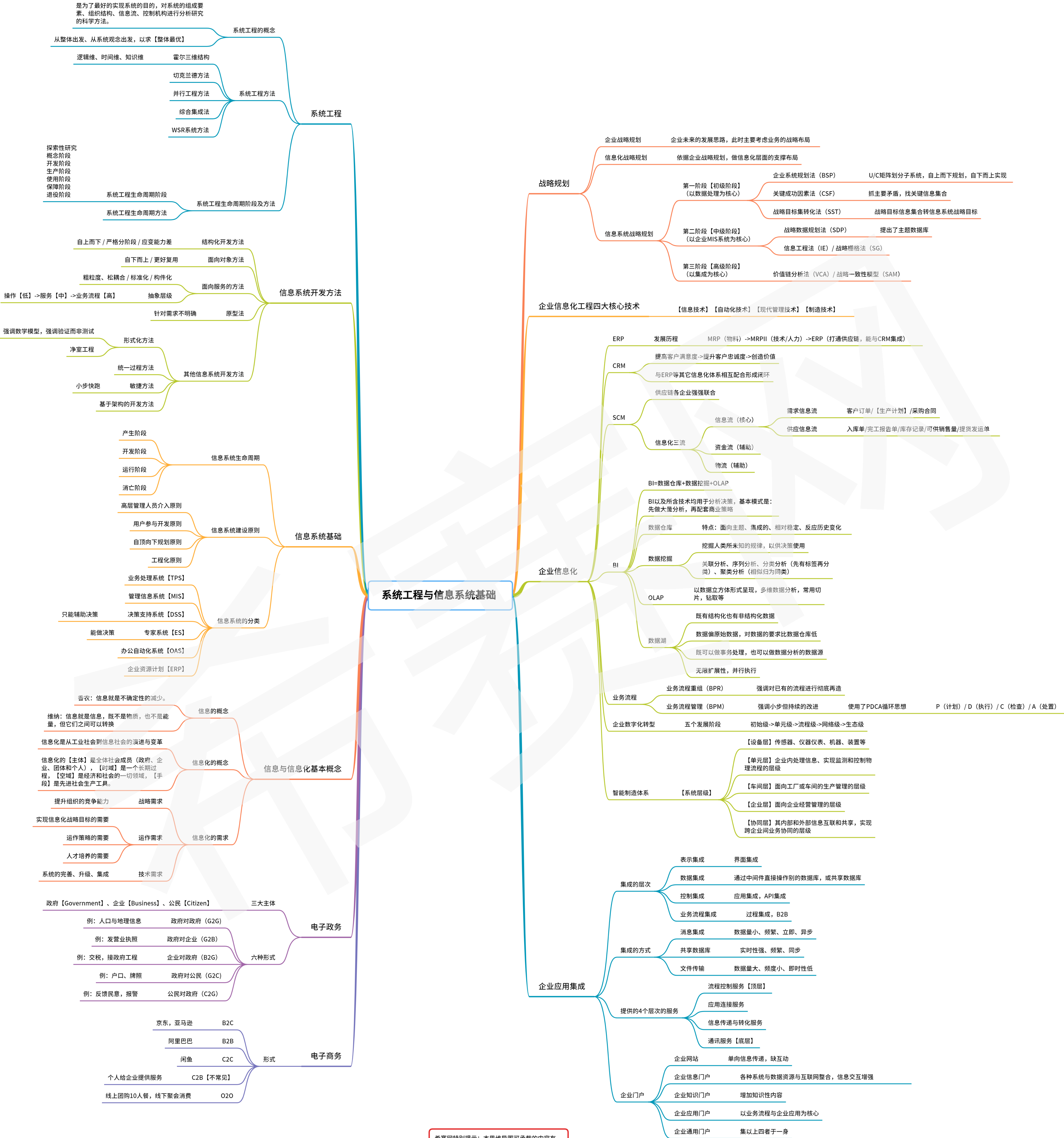
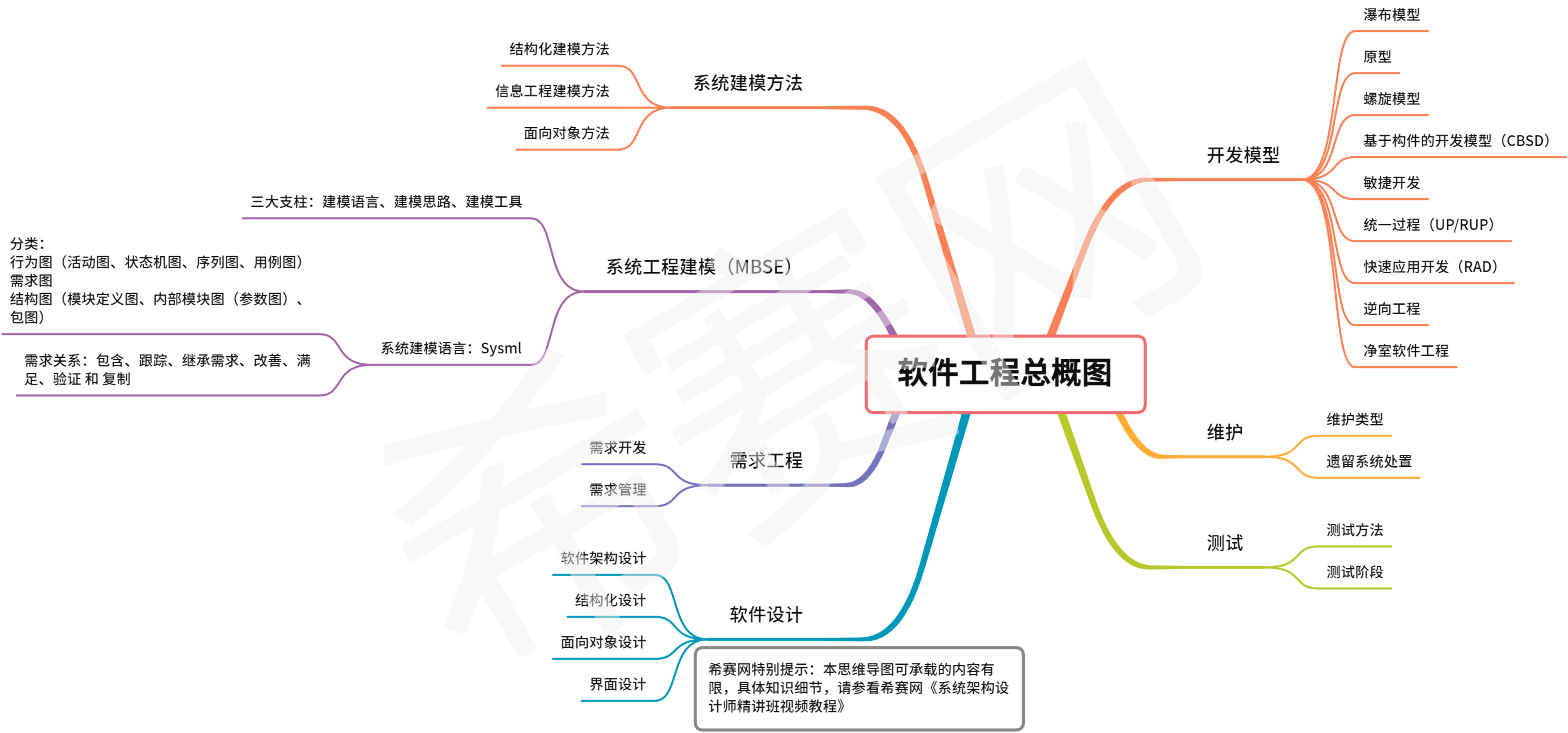
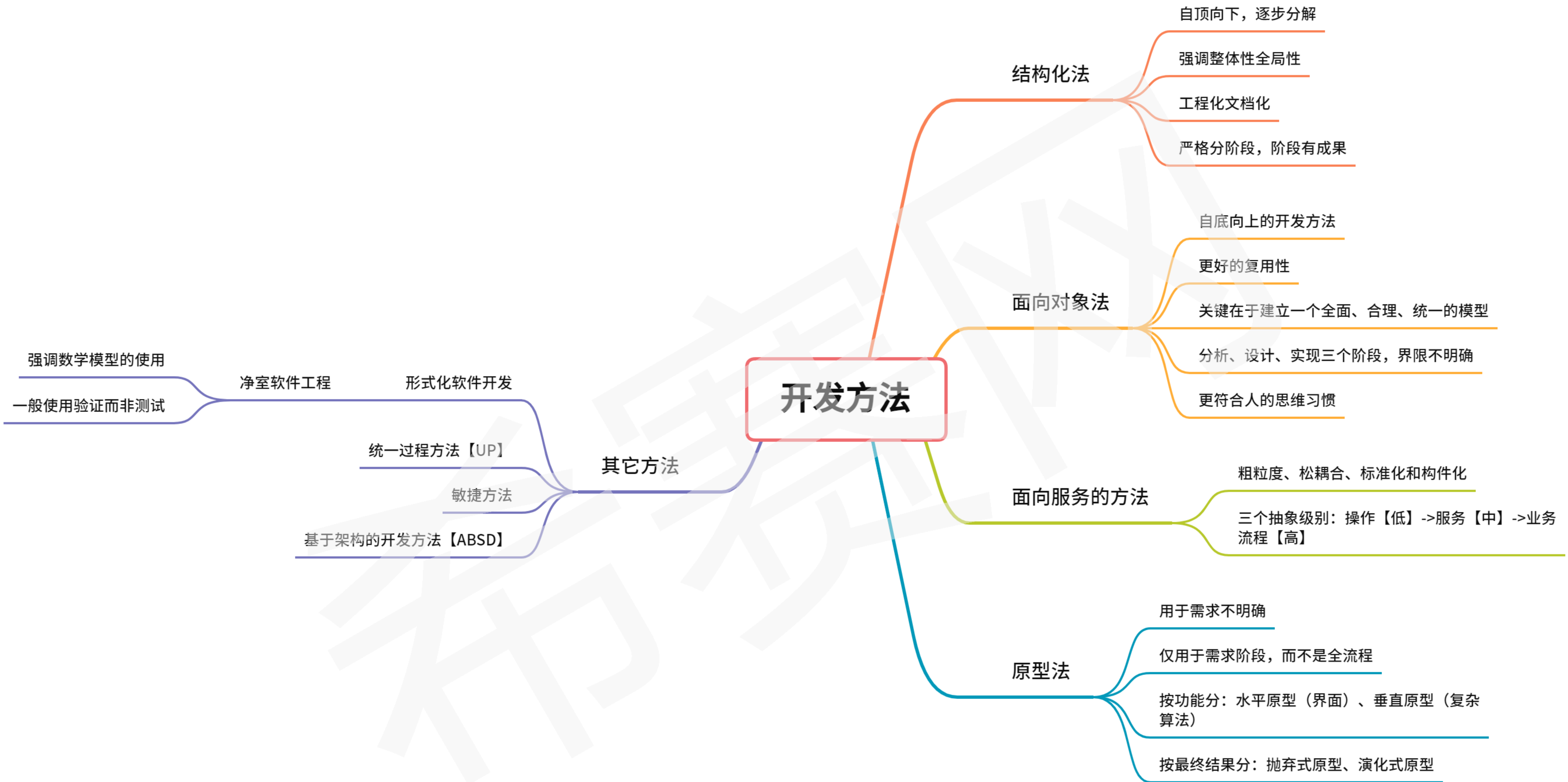


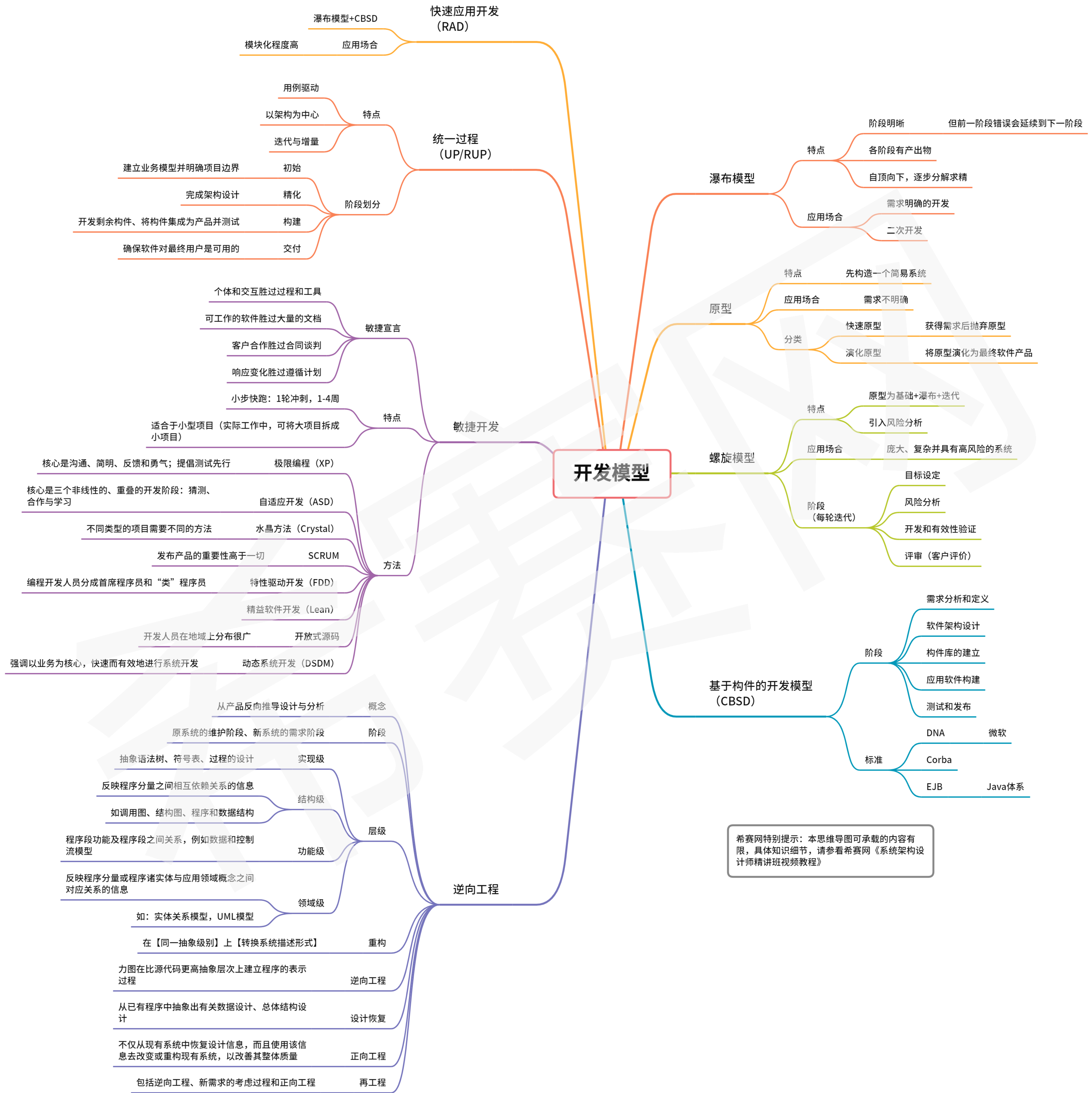




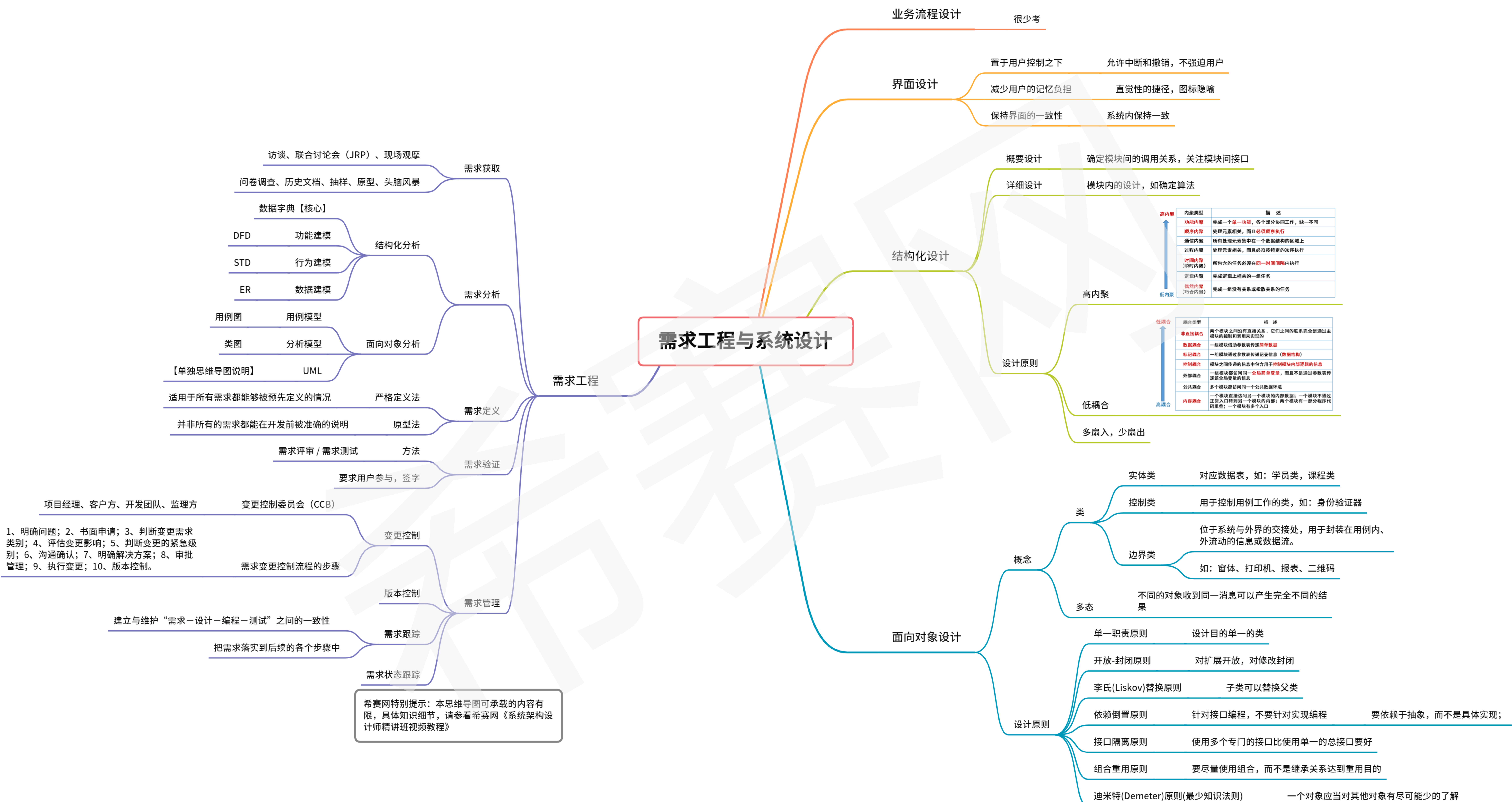


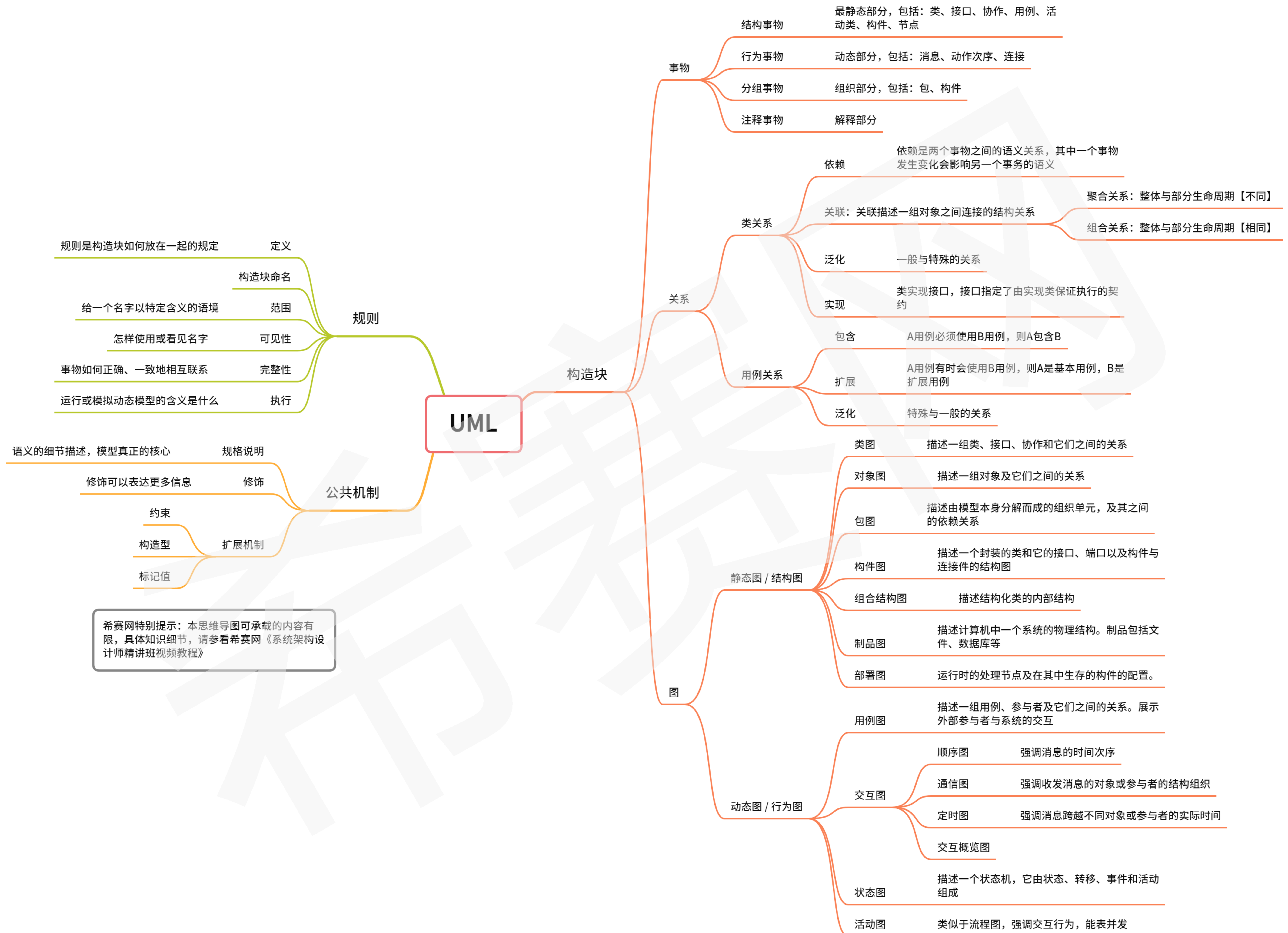


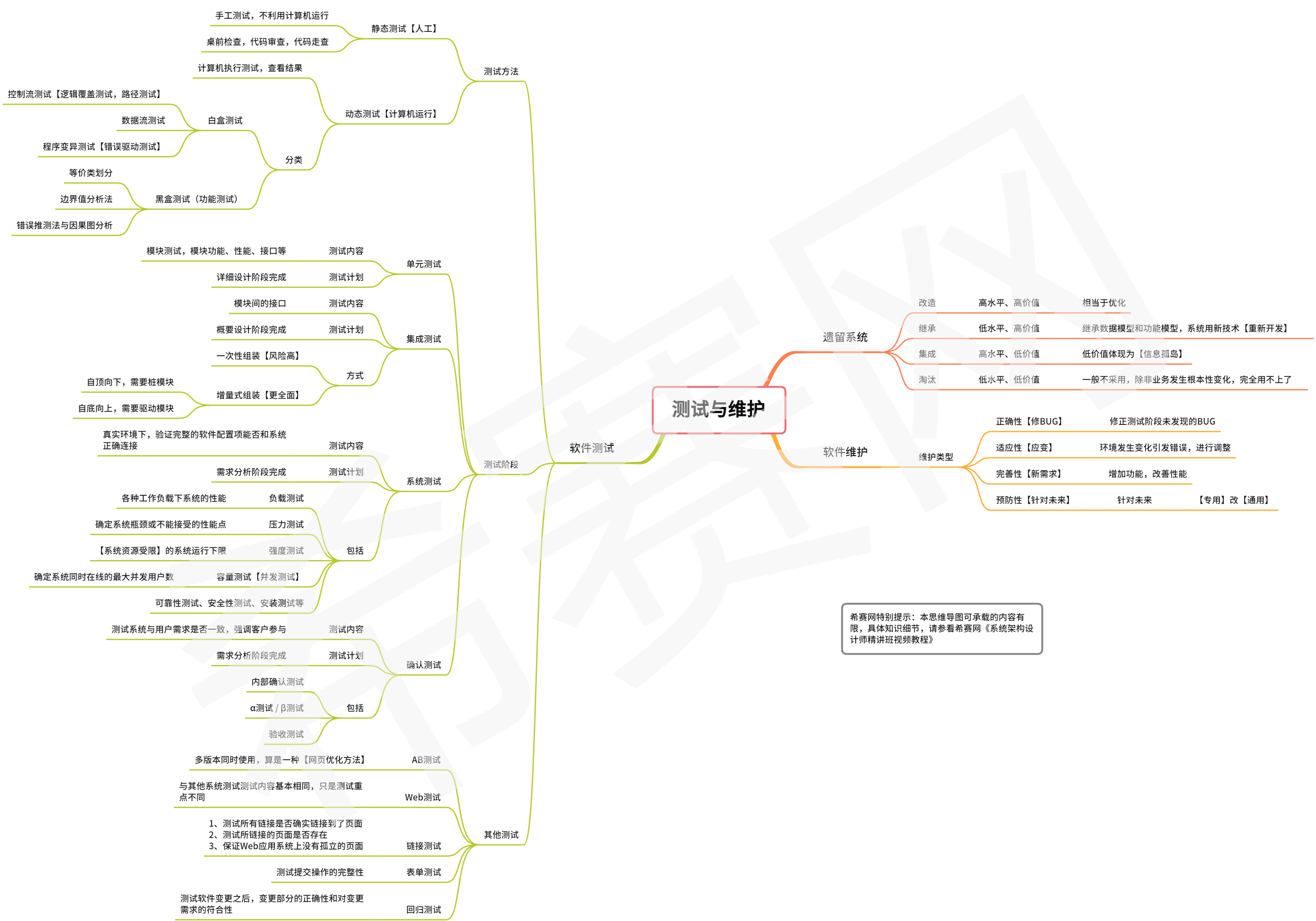
希赛网特别提示：本思维导图可承载的内容有限，具体知识细节，请参看希赛网《系统架构设计师精讲班视频教程》



需求工程与系统设计







软件架构 (1/3)

架构概念

- 所处阶段需求之后，设计之前
- 核心任务将需求分配到各个构件上
- 定义1软件系统提供了一个结构、行为和属性的高级抽象
- 定义2干系人进行交流的手段
- ADL
 - 架构的组成：构件、连接子、架构配置（三要素）
 - 典型的ADL：
 - C2SADL【基于组件和消息的软件架构描述语言】
 - Wright【分布、并发类型的架构描述语言】
 - ACME【架构互换语言】
 - UniCon【基于组件和连接的架构描述语言】
 - Rapide【基于事件的架构描述语言】
 - 其他【Darwin、MetaH、Aesop、Weaves、SADL、xADL】

软件架构复用

- 类型
 - 机会复用开发过程中，只要发现有可复用资产，就对其进行复用
 - 系统复用开发之前，要进行规划，以决定哪些需要复用
- 基本过程复用的基本过程主要包括3个阶段：首先构造/获取可复用的软件资产，其次管理这些资产，最后针对特定的需求，从这些资产中选择可复用的部分，以开发满足需求的应用系统

定义

描述系统组织方式和惯用模式，反映多系统共有的结构和语义特征，强调架构设计复用

架构风格

- 数据流风格
 - 共性交互差；分阶段处理数据，上阶段的结果是下一阶段的输入
 - 批处理风格数据必须以整体方式传递
 - 管道过滤器风格
 - 优点良好的复用性
 - 可维护性和可扩展性较强
 - 支持并发
 - 经典实例传统编译器、网络数据包处理
 - 调用/返回风格
 - 主程序/子程序
 - 面向对象
 - 优点高度模块性、封装功能、代码共享、灵活性、易维护性、可扩充性
 - 缺点调用时需要知道对方对象ID，这样会产生紧耦合
 - 分层
 - B/S、C/S
 - M（模型）EJB
 - V(视图)JSP
 - C（控制）Servlet
 - MVP
 - MVVM
 - 优点利于复用；每一层只影响两层
 - 缺点分层不易；层次多了影响性能
 - 设计时需要注意的点：1、污水池反模式。（请求简单穿透多层，但没做业务逻辑。）2、分层架构可能会让应用变得庞大。
- 独立构件风格
 - 共性不直接调用
 - 进程通信消息传递
 - 事件驱动系统（隐式调用）不直接调用，通过触发事件进行
 - 经典实例现代集成编译环境（联动机制）、订阅-发布
- 虚拟机风格
 - 共性构建自定义语言的执行引擎
 - 解释器
 - 基于规则的系统人工智能、DSS
 - 经典实例JVM、工作流引擎、自定义游戏关卡
- 仓库风格
 - 共性以数据为中心，其它职能围绕数据展开
 - 数据库系统
 - 黑板系统信号处理、语音识别、问题规划等复杂问题
 - 超文本系统
 - 经典实例现代集成编译环境（以语法树为中心）
- 其它风格
 - 过程控制（闭环）
 - 反馈机制形成闭环
 - 经典实例空调控制、定速巡航
 - 模型驱动架构（MDA）
 - 计算独立平台（CIM）->平台独立模型（PIM）->平台相关模型（PSM）->代码(Code)
 - C2架构风格
 - 构件和连接件都有一个顶部和一个底部。构件之间不允许直连。一个连接件可以和任意数目的其他构件和连接件连接。当两个连接件进行直接连接时，必须由其中一个的底部到另一个的顶部。

基于架构的开发方法 (ABSD)

- 特点
 - 架构驱动
 - 强调使用视角与视图描述架构
 - 强调由业务、质量和功能需求的组合驱动架构设计
 - 使用用例描述功能需求
 - 使用质量场景描述非功能需求
- 三个基础
 - 功能的分解
 - 通过选择架构风格来实现质量和业务需求
 - 软件模板的使用
- 开发过程 (迭代式)
 - 架构需求
 - 需求获取
 - 标识构件
 - 需求评审
 - 架构设计
 - 提出架构模型
 - 映射构件
 - 分析构件相互作用
 - 产生架构
 - 设计评审
 - 架构文档化
 - 文档编写原则
 - 使用者的角度进行编写
 - 必须分发给所有与系统有关的开发人员
 - 输出成果
 - 体系结构规格说明
 - 测试体系结构需求的质量设计说明书
 - 架构复审
 - 目的标识潜在的风险，及早发现架构设计中的缺陷和错误
 - 内容
 - 质量需求是否在设计中得到体现
 - 体系结构能否满足需求
 - 层次是否清晰
 - 构件的划分是否合理
 - 构件的设计是否满足功能与性能的要求
 - 文档表达是否明确
 - 架构实现
 - 流程分析与设计->构件实现->构件组装->系统测试
 - 架构演化
 - 应用场景
 - 开发过程中需求变动
 - 交付后需求变动
 - 流程需求变动归类->制订架构演化计划->修改、增加或删除构件->更新构件的相互作用->构件组装与测试->技术评审

希赛网特别提示：本思维导图可承载的内容有限，具体知识细节，请参看希赛网《系统架构设计师精讲班视频教程》

